Hochschule Fulda
University of Applied Sciences

Global Distributed Software Development - AI5015 / AI5088

Team Project Summer 2025 - Milestone 2

Fulda Einkaufszentrum

(Hochschule Fulda Internal Marketplace for Students and Staff to Buy/Sell/Exchange goods.)

(Dated: 22/05/2025)

Meet the team: **Team 4**

Name	Email	Role
Pavithra Purushothaman	pavithra.purushothaman@informatik.hs-fulda.de	Team Lead
Akhil Baiju	akhil.baiju@informatik.hs-fulda.de	Database Engineer
Mufidha Sulfikerali	mufidha.sulfikerali@informatik.hs-fulda.de	GitHub Master
Udit Parihar	udit.parihar@informatik.hs-fulda.de	Backend Developer
Ayush Kaushik	ayush.kaushik@informatik.hs-fulda.de	Frontend Lead
Ridhima Tigadi	ridhima.tigadi@informatik.hs-fulda.de	Backend Lead

Supervisor: Prof. Dr. Rainer Todtenhöfer

Table of Contents

- Milestone 1 - Revised Version
- Milestone 2
    1. Functional Requirements - Prioritized
    2. List of Main Data Items and Entities
    3. UI Mockups and Storyboards
    4. High-Level Architecture, Database Organization
    5. High-Level UML Diagrams
    6. Identify Actual Key Risks
    7. Project Management

Milestone 1 - Revised Version

1. Executive Summary

Motivation & Importance:

Many students are arriving at the Hochschule Fulda every semester but not all of them are successful in getting basic living essentials at the very beginning of their studies. On the other hand, there are students graduating every semester looking to declutter/sell their used items. However, there is no safe platform for them to perform this buy/sell/exchange within themselves in the Fulda community. There are still different platforms like Kleinanzeigen, eBay and Facebook Marketplace available online but they are not too relevant and trustable. The main aim of this project is to provide a safe and easy place for students, staffs in the Hochschule Fulda community to buy/sell/exchange items with each other directly.

Key Functions & Services:

- Signup/Login with Fulda Email
- Create or View Ads
- Search, Sort, and Filter
- Admin Approval Before Posting
- My Ads Section
- Report a Spam Ad
- Pickup Spot Suggestions
- Seller's Known Languages
- Ad Expiry Limit (180 Days)
- User Expiry for Inactive Accounts
- Campus-Only Community
- Product Recommendation
- Contact via Chat
- Auto Spam detection
- Buyer recommendations to Seller
- Wishlist in the app
- Notification for Requested ads
- Bulk import Ads
- Audio search

Unique Value Proposition:

The application was created specifically for the Fulda community, that's why this makes it unique. Only verified university members can buy products, keeping members safe and posts relevant. The experience will be easy and continuous with unique features like Auto spam detection, Buyer recommendations to Seller, Notification for Requested ads, Bulk import Ads, Automatic listing expiration, Seller known languages and suggested pickup locations. This is more than just an app; it was created with Fulda communities unique needs and habits in mind.

Team Overview:

Our team members have a variety of backgrounds, few with technical experience, project management skills and also some with product development expertise. We have Pavithra with project management skills who keeps the team focussed on the project goals, initiating team meetings, adding work items in the collaboration tools, also addressing both the technical and non-technical issues within the team along the way and also the point of contact for communicating with CTO/CEO. We have backend developers Ridhima, Udit who address the challenges we face in the development of the application with their technical expertise. We have Mufidha who is our deployment engineer who takes care of our GitHub CI/CD deployment with her cloud experience. We have Ayush who is our experienced front end developer who has keen focus on UI development to make the application user friendly and better to the users. We also have Akhil who is our Database Engineer who takes care of the application data in the backend to make sure it is in a well defined structured manner for processing.

2. Personas and Main Use Cases

2.1 Personas:

Persona 1: Lillian Becker – The Student Seller (Primary Persona)

- **Characteristics:** A fourth-year student preparing for graduation, Lillian wants to declutter and earn extra cash by selling unneeded items like textbooks.
- **Goals:** Sell used textbooks and other student supplies quickly; ensure smooth, hassle-free transactions within campus.
- **Skills:** Familiar with uploading listings and responding to messages promptly.
- **Pain Points:** Time constraints during finals; doesn't want to deal with strangers or leave campus to complete transactions.

Persona 2: Yash Gupta – A Student Buyer(Primary Persona)

- **Characteristics:** A new international graduate student unfamiliar with local shopping and cultural norms.
- **Goals:** Furnish his dorm room affordably and quickly; connect with sellers who understand his language or context.
- **Skills:** Tech-savvy and fluent in English; adapts easily to online buying platforms.
- **Pain Points:** Cultural and logistical unfamiliarity; limited local contacts; potential language barriers.

Persona 3: Lukas Brandt – The Guest User

- **Characteristics:** Lukas is an alumnus visiting the university campus. He's curious about the platform and wants to explore available listings but doesn't have a university login.
- **Goals:** View available products and services without account creation; get a general idea of item prices and availability.
- **Skills:** Comfortable browsing web platforms with basic navigation skills.

- **Pain Points:** Cannot contact sellers or save listings without a verified student/staff login; limited interaction options.

Persona 4: Admin Role – The Platform Moderator

- **Characteristics:** A designated campus administrator monitoring the platform's content and users.
- **Goals:** Keep listings appropriate and relevant; remove scams or flagged users; oversee analytics and maintenance.
- **Skills:** Strong backend and moderation experience; familiar with data dashboards and website operations.
- **Pain Points:** High volume of listings and users to review; balancing control with user autonomy.

2.2 Use Cases:

UC-1: Student Sells Textbooks After Semester

- **Actor:** Lillian Becker (Seller)
- **Goal:** Sell textbooks to other students.
- **Description:** Lillian posts listings, receives interest from a first-year student, and meets at the campus library for a smooth handover.

UC-2: Student Furnishes Dorm Room

- **Actor:** Yash Gupta (Buyer)
- **Goal:** Buy essential furniture affordably.
- **Description:** Yash browses furniture listings, contacts multiple sellers, and picks up the items from different locations in the campus over a few days.

UC-3: Guest Views Public Listings

- **Actor:** Lukas Brandt (Guest)
- **Goal:** Explore available listings without signing up.
- **Description:** Lukas browses the platform's public listings (e.g., titles, categories, and photos). However, he cannot view full details, contact sellers, or post listings until he registers with a Fulda email ID.

UC-4: User Flags a Scam User

- **Actor:** Yash Gupta (Buyer)
- **Goal:** Report a suspicious user.
- **Description:** After noticing suspicious behavior, they use the "mark as scam" feature. If a user is flagged repeatedly, the system highlights the user for admin review.

UC-5: Admin Deletes Inappropriate Listings

- **Actor:** Admin Role
- **Goal:** Maintain listing quality.
- **Description:** Admin identifies inappropriate or scam content and removes it from the platform after review.

UC-6: Admin Monitors Dashboard and Reports

- **Actor:** Admin Role
- **Goal:** Track activity and maintenance needs.
- **Description:** Admin checks user and posting analytics, receives bug reports, and flags items for the developer to address.

UC-7: Buyer Contacts Seller for Product Inquiry via chat messaging

- **Actor:** Yash Gupta (Buyer)
- **Goal:** Clarify product details before making a purchase.
- **Description:** Yash finds a listing for a used monitor and wants more information about its condition and warranty. He uses the platform's messaging feature to contact the seller, asks a few questions, and arranges a time to inspect the item before deciding to buy.

3. List of Main Data Items and Entities

Below is a table defining key data items and entities used in your application. Including columns for Index, Entity/Item Name, and Description (Purpose/Usage).

Index	Entity/Item Name	Description (Purpose/Usage)
1	User	Used for authentication, authorization (Student, Staff, Faculty).
2	Item	Object listed for sale or exchange. Used in browsing and searching.
3	Message	In-platform communication between Buyers and Sellers.

4	Category	Helps users to browse and filter items based on their needs.
5	Report	Allows Admins to check and review listings and user behavior.
6	Approval Queue	Ensures Admin approves before listings become public.
7	Media Asset	Enriches item visibility with visual content.
8	Login Session	Tracks user activity and helps enforce usage limits.

4. Initial List of Functional Requirements

1. **User Registration and Login**

The system shall allow users to register and log in using university email addresses to ensure campus-only access.

2. **Campus Registration**

Verify users' university affiliation through email domain to ensure campus-only access.

3. **Product Browsing and Search**

Users shall be able to browse and search for available products using filters like category, price, location.

4. **Create New Product Listings**

Users shall be able to create new listings by adding images, descriptions, price, location, and whether the item is for sale or swap.

5. **Time-Limited Listings and User Access**

Users can set product listing expiry up to 180 days; expired listings are auto-archived unless renewed. User accounts (e.g., guests) expire after 180 days unless extended by admin.

6. **Edit or Delete Listings**
Enable sellers to update or remove their listings at any time before the item is sold or expired.

7. **View Product Details**
Users shall be able to view detailed information about a product, including description, price, images, seller info, and location.

8. **Chat based communication**
If users have queries related to a purchase or sale, the system shall trigger an email to the respective buyer or seller to initiate communication.

9. **Swap Offers**
Users shall be able to propose swaps (item-for-item deals) instead of paying with money.

10. **Role-Based Access Control**
System distinguishes between students, faculty, guests, and admins with appropriate feature access.

11. **Pickup or Meetup Coordination**
The system shall suggest nearby pickup locations on campus and allow users to select or propose meeting points.

12. **User Profile**
Each user shall have a profile page displaying their listings, requests, swaps.

13. **Display Seller's Known Languages**
The system shall allow sellers to indicate the language they prefer, i.e. German or English, which will be displayed on their profile or listing to help buyers choose an appropriate communication language.

14. **Report Spam or Inappropriate Content**
Users shall be able to report listings or profiles they suspect to be spam, or inappropriate; reported content will be flagged for review.

15. **Google Analytics Integration**
Tracks user behavior, most viewed products, session times.

5. List of Non-Functional Requirements

1. Application shall be developed, tested and deployed using tools and servers approved by Class CTO and as agreed in Milestone 0. Application delivery shall be from chosen cloud server
2. Application shall be optimized for standard desktop/laptop browsers e.g. must render correctly on the two latest versions of two major browsers
3. All or selected application functions must render well on mobile devices

4. Data shall be stored in the database on the team's deployment cloud server.
5. No more than 50 concurrent users shall be accessing the application at any time
6. Privacy of users shall be protected and all privacy policies will be appropriately communicated to the users.
7. The language used shall be English (no localization needed)
8. Application shall be very easy to use and intuitive
9. Application should follow established architecture patterns
10. Application code and its repository shall be easy to inspect and maintain
11. Google analytics shall be used (optional for Fulda teams)
12. No email clients shall be allowed.
13. Pay functionality, if any (e.g. paying for goods and services) shall not be implemented nor simulated in UI.
14. Site security: basic best practices shall be applied (as covered in the class) for main data items
15. Application shall be media rich (images, video etc.). Media formats shall be standard as used in the market today
16. Modern SE processes and practices shall be used as specified in the class, including collaborative and continuous SW development
17. For code development and management, as well as documentation like formal milestones required in the class, each team shall use their own GitHub to be set-up by class instructors and started by each team during Milestone 0
18. The application UI (WWW and mobile) shall prominently display the following exact text on all pages "Fulda University of Applied Sciences Software Engineering Project, Summer 2025 For Demonstration Only" at the top of the WWW page. (Important so as to not confuse this with a real application).

6. Competitive Analysis

6.1 Competitive Analysis Table:

Below is a table comparing key features of competitor products versus your planned product.

Features	Einkaufszentrum	Ebay	Kleinanzeigen	Facebook Marketplace
Signup/Login	✅	✅	✅	✅
Create or View Ads	✅	✅	✅	✅
Search, Sort, and Filter	✅	✅	✅	✅
Admin Approval Before Posting	✅	❌	❌	❌
Pickup Spot Suggestions	✅	❌	❌	❌
Campus-Only Community	✅	❌	❌	❌
User Expiry for Inactive Accounts	✅	❌	❌	❌
Contact via Chat	✅	✅	✅	✅
Ad Expiry Limit (180 Days)	✅	❌	❌	❌
Unique Selling Points:				
Bulk import Ads	✅	❌	❌	❌
Notification for Requested ads	✅	❌	❌	❌
Product Recommendation	✅	✅	✅	✅
Buyer recommendations to Seller	✅	❌	❌	❌
Auto Spam detection	✅	❌	❌	❌
Bundled Product recommendations	✅	❌	❌	❌
Audio search	✅	❌	❌	❌

6.2 Analysis Summary:

In contrast to widely used platforms such as eBay, Kleinanzeigen and Facebook Marketplace, our application is designed from the ground up for the Fulda University community. All of these platforms enable users to post and browse items, but our product was designed to offer a safer and more focused platform. Our Unique Selling Points have all been built into our application 's features to ensure Fulda students and staff use and trust the application. Also, the fact we have

a button to indicate which languages a seller can speak, will hopefully allow users once again break the communication barrier. The design of our application ensures every post as well as a user interface is free of ads, so users can focus on what matters to them. We have real time chat in the app for reaching out to a seller which keeps things simple while protecting a user's privacy. Overall this is not just another marketplace , it is a product intended and deeply thought out to help build a platform for the campus community by using features designed from real campus needs.

7. High-Level System Architecture and Technologies Used

Technologies

• Server Host: AWS EC2 t2.micro (1 vCPU, 1 GB RAM)

• Operating System: Ubuntu 22.04 LTS

• Database: MySQL 8.0

• Web Server: NGINX 1.22

• Server-Side Language: Python 3.10

• Web Framework: Flask 2.3

• Frontend Library: React 18

• Cloud Storage: AWS S3

• IDE: VSCode

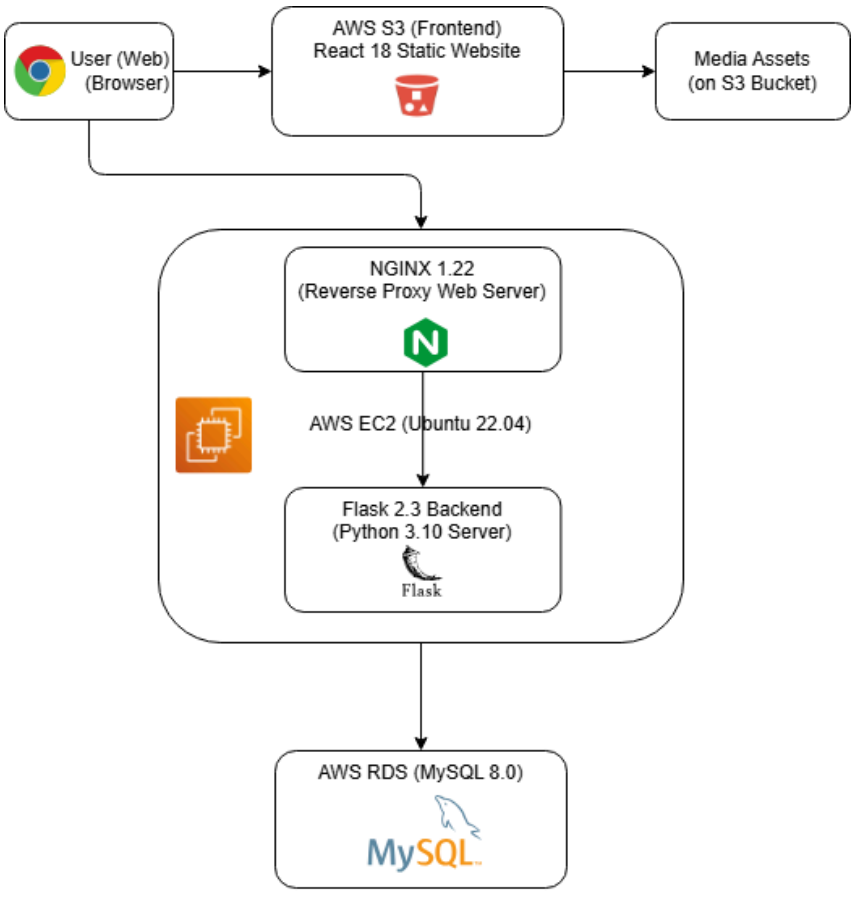


Fig: Web App Deployment Architecture

Deployment Information

- Supported Browsers: Chrome 130+, Firefox 132+, Edge, Safari, Brave (not IE)
- Deployment Method: Automated via GitHub Actions.
- How It Works:
- The workflow is triggered on every push to the main branch.
- It checks out the source code from the repository.
- AWS credentials are securely configured using GitHub Secrets.
- The files are synchronized to an AWS S3 bucket using the AWS CLI.
- Any files removed from the repository are also deleted from the S3 bucket to keep it in sync.
- Hosting: The static website is served directly from the configured S3 bucket endpoint.

8. Team and Roles

Team's roles and responsibilities of this project.

Name	Role	Responsibilities
Pavithra Purushothaman	Team Lead	Keep the team focussed on the project goals and to communicate with CTO/CEO
Akhil Baiju	Database Engineer	Maintains and organises project database
Mufidha Sulfikerali	GitHub Master	Organizes Git and makes sure git guidelines and good practices are followed
Udit Parihar	Backend Developer	Backend Development
Ayush Kaushik	Frontend Lead	Review frontend Pull Requests
Ridhima Tigadi	Backend Lead	Review backend Pull Requests

9. Checklist

Checklist for team organisation with DONE/ ON TRACK/ ISSUE status

Task Item	Status
Finalised meeting schedule	DONE
GitHub Master chosen	DONE
Agreed on Tech Stack	DONE
Learning approved tech stack	ON TRACK
Team's understanding of Milestone 1	DONE
GitHub repository organized	DONE

.

Milestone 2:

1. Functional Requirements - Prioritized

Priority 1 Requirements:

Unregistered Users

1. User Login

1.1: Users will authenticate with their valid university email addresses for campus access only.

1.2: There should be a way for users to log in and out safely.

2. Campus Registration & Access

2.1: User must have their email verified, from known university domains.

2.2: Restrictions and need for admin renewal for guest users after 180 days on the service.

Registered Users

3. Product Browsing and Search

3.1: Users can search for items through categories, price, location and keywords.

3.2: Listings will be categorized and have filters associated with it similar to the ("Textbooks", "Dorm Room" etc.).

4. Create New Product Listings

4.1: Sellers can add title, description, images, category, price, location and swap/buy tag.

5. Time-Limited Listings and Account Lifecycle

5.1: The maximum duration for listings is 180 days.

5.2: Guest accounts are automatically deleted after 180 days, unless the account is validated by an admin.

6. Edit or Delete Listings

6.1: Vendors are also allowed to edit or remove listings if it's sold or expired.

7. View Product Details

7.1: This will list the product details which include description, price, location, images, seller info

8. Bulk Import for Power Sellers

8.1: Sellers/Admins can upload list of items in batch using CSV/Excel.

8.2: Each entry is system parsed and validated prior to being posted.

9. Chat-Based Buyer-Seller Communication

9.1: There will be communication between buyer and seller to buy and sell products.

10. Swap Offers

10.1: The possibility for buyers to propose exchange of another item instead of payment.

10.2: Sellers can approve, refuse or propose a counter offer.

11. Admin Approval

11.1: Admins can view user activity, review spam and listings.

11.2: Admins can flag,or remove inappropriate content.

12. Report Spam or Inappropriate Content

12.1: Users can report listings or profiles suspected to be spam.

12.2: Flagged content appears in the admin moderation queue.

13. Request Based Deal Alerts

13.1: Notify users when a requested product is available from a seller.

14. Wishlist

14.1: Buyers can save listings to a personal wishlist.

15. Google Analytics Integration

15.1: Track user behavior, session durations, most viewed items, and traffic sources.

Priority 2 Requirements:

16. Auto Spam Detection

16.1: System scans listings for spam-like behavior.

17. Personalized Product Recommendations

17.1: Recommendations will be displayed for certain products based on user history, search terms, and category interest.

18. Buyer Recommendations to Sellers

18.1: Sellers can view a list of users who've recently searched for matching keywords.

Priority 3 Requirements:

Registered Users

19. Bundled Product Suggestions

19.1: Suggest related or complementary items often bought together.

20. Text-to-Speech for Product Listings

20.1: Converts listing description to audio for visually impaired users.

2. List of Main Data Items and Entities

This section defines key logical entities and their attributes at a high level. The application relies on the following core data entities, represented as database tables in MySQL:

1. USER: Authentication, access control, user identification

• Fields:

- User ID (PK), Full Name, Email (@hs-fulda.de)

- Password (hashed), Role (User,, Admin)

- Date Joined, Profile Status (Active/Banned)

- Optional: Contact Number, Department

2. ITEM: Products listed by users

• Fields:

- Item ID (PK), Title, Description, Price, Category ID (FK)

- Seller ID (FK), Date Posted, Status (Active/Sold/Deleted/Rejected)

- External Media URL, Location (optional)

3. MESSAGE: Buyer and Seller communication

• Fields:

- Message ID (PK), Sender ID (FK), Receiver ID (FK), Timestamp

- Content, Item ID (FK), Read Status

4. CATEGORY: Item filtering

• Fields:

- Category ID (PK), Category Name (e.g., Books, Electronics)

- Description

5. REPORT: Monitor inappropriate content

• Fields:

- Report ID (PK), Reported Item/User ID (FK), Reporter ID (FK)

- Timestamp, Reason, Status (Pending/Reviewed/Action Taken)

6. APPROVAL QUEUE: Admin review of new listings

• Fields:

- Queue ID (PK), Item ID (FK), Submitted By (User) (FK), Submission Time

- Approval Status, Admin Reviewer ID (FK)

7. MEDIA ASSET (Cloud-hosted): External media reference

• Fields:

- Media ID (PK), Item ID (FK), Media Type (Image/Video)

- External URL, Thumbnail URL, Description (optional)

8. LOGIN SESSION: Track access & usage

• Fields:

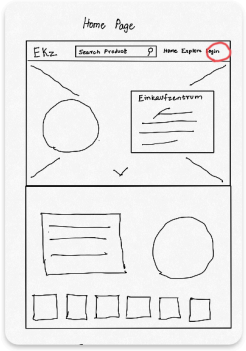
- Session ID (PK), User ID (FK), Login & Logout Timestamps

- Optional: IP Address, Device Info

3. UI Mockups and Storyboards

Use case 1

Student sells text books after semester

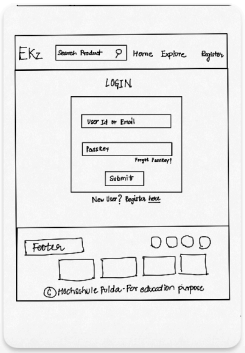


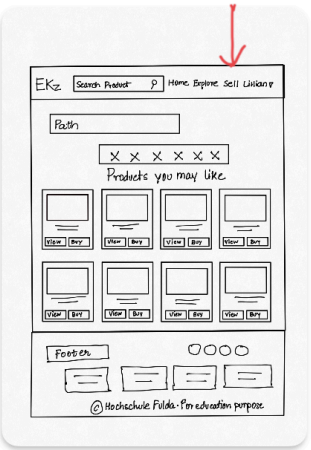
Screen 1: Homepage

- **Scene**: Lillian opens the Einkaufzentrum website.
- **Action**: She clicks on the **"Login"** button in the top-right corner.

Screen 2: Login Page

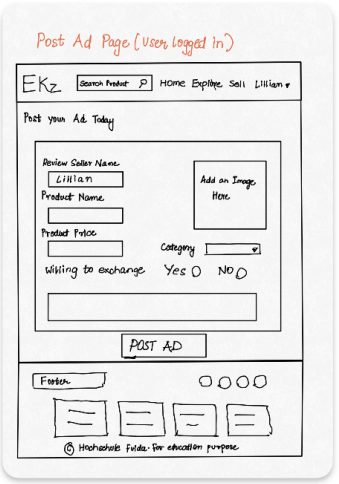
- **Scene**: The login form pops up.
- **Action**: Lillian types in her username and password and clicks **"Login"**.





Screen 3: Explore Page

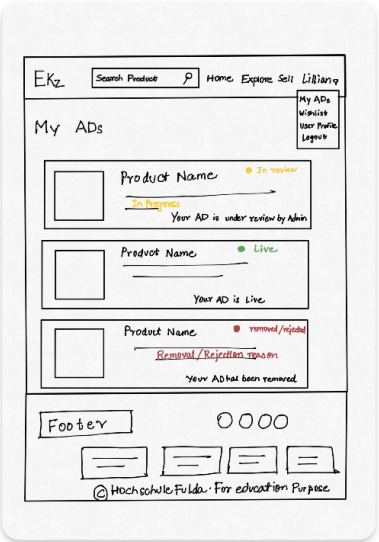
- **Scene**: After successfully logging in, Lillian finds herself on the Explore page.
- **Action**: She clicks on the **"Sell"** button (in the navigation bar) to add a new post.



Screen 4: Add New Post Page

- **Scene**: A pop-up window appears to add a book listing.
- **Action**:

- Lillian grabs an image of her book and adds it to the form.
- She fills in the details like the book's name, edition, author, year, and a short description.
- She double-checks everything to make sure it's all correct.
- Now she has an option to either sell the book or exchange the book with the buyer's offering by selecting yes or no option.
- Finally, she clicks the **"Post Ad"** button.

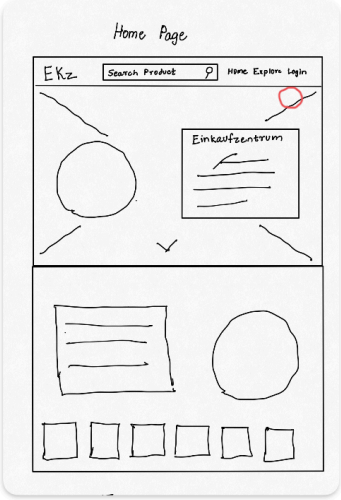


Screen 5: My ADs Page (Post Submission Confirmation)

- **Scene**: Lillian gets redirected back to My ADs page.
- **Action**: She checks the status of her AD and gets updated about the action taken by the administrator on her AD (In review, Live, Rejected/Removed)

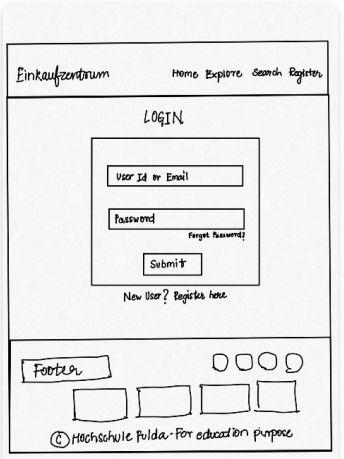
Use case 2

Student wants to buy a laptop



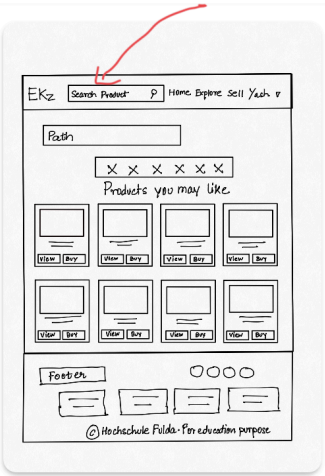
Screen 1: Homepage

- **Scene**: Yash opens the Einkaufzentrum website.
- **Action**: He clicks on the **"Login"** button.



Screen 2: Login Page

- **Scene**: The login form pops up.
- **Action**: Yash types in his username and password and logs in.



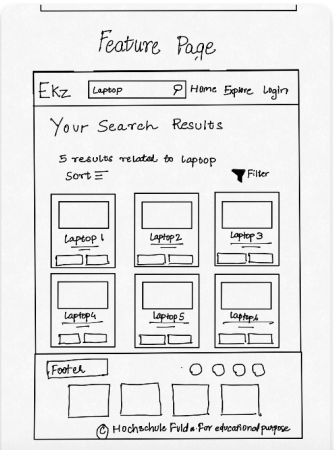
Screen 3: Explore Page

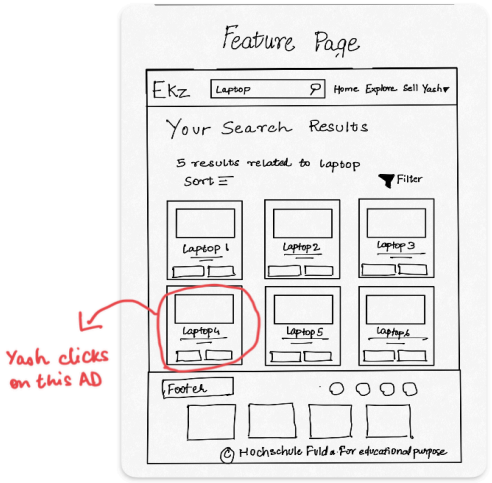
- **Scene**: After successfully logging in, Yash is greeted with the Explore page.
- **Action**: He clicks on the **"Search"** button to start looking for specific products.

Screen 4: Search Page

- **Scene**: The search bar is now active

- Yash types in a laptop with specific requirements as per his choice..
- He presses **Enter** to initiate the search.





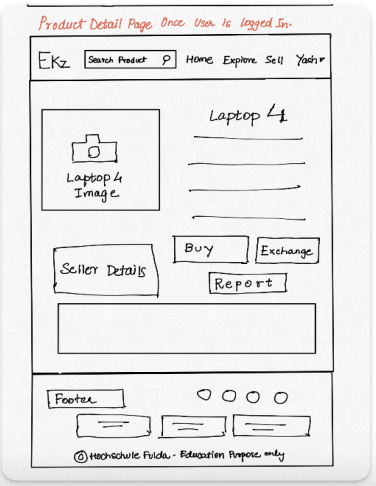
Screen 5: Search Results Page

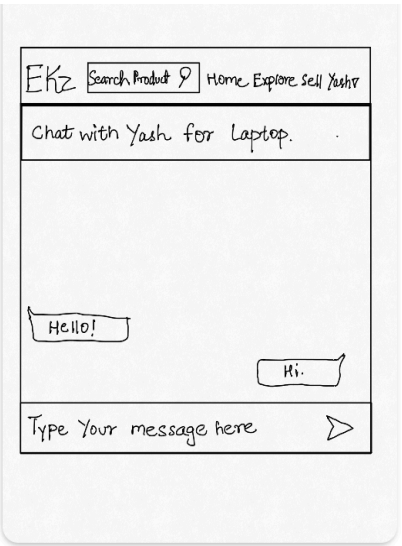
- **Scene**: The system shows a list of laptop ads that match his search.
- **Action**: Yash looks through the listings and finds one that matches his expectations

Screen 6: Product Details Page

- **Scene**: Yash clicks on the post he's interested in to see all the details. After checking it out, he decides to buy it.
- **Action**:

- He reviews all the seller-provided details, including images, pricing, condition, and description.





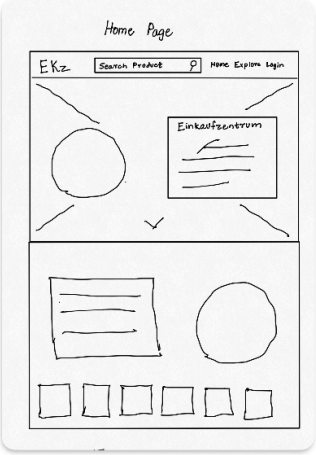
Screen 9: Messaging Initiation

- **Scene**: After deciding to go ahead with the purchase, Yash clicks the **"Buy"** button.
- **Action**:

- He's taken to the built-in **messaging page**, where he can **chat** directly **with the seller** to discuss the details and finalise the deal.

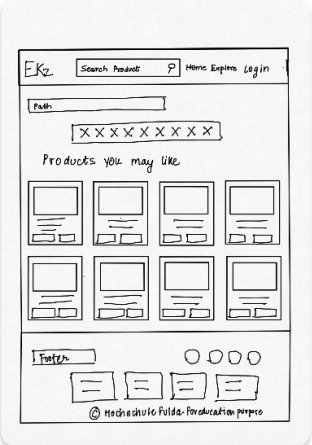
Use case 3

Guest Views Public Listings



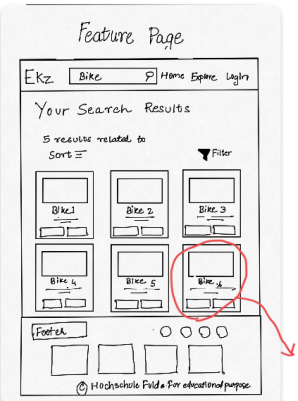
Screen 1: Homepage

- **Scene**: Lukas opens the Einkaufzentrum website and lands on the homepage.
- **Action**: He clicks on the **"Explore"** button to start browsing listings as a guest.



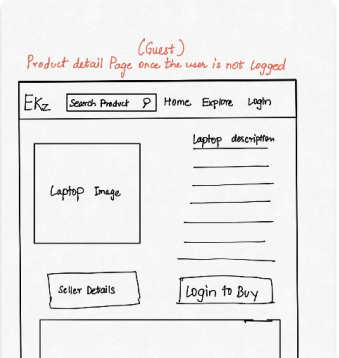
Screen 2: Explore Page

- **Scene**: The Explore page pops up with a bunch of listings.
- **Action**: Lukas takes a look and finds some Bikes. He's thinking about making a more specific search.
- Lukas clicks on the **"Search"** area and types in bike and hit enter.



Screen 3: Search Results Page (Initial)

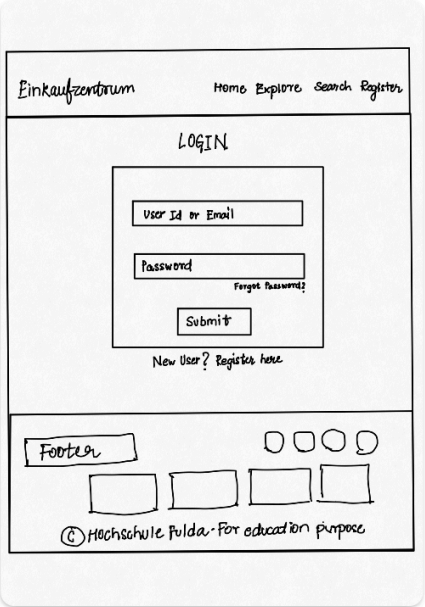
- **Scene:** Lukas is directed to the **Search Results** page.
- **Action:**
- All pertinent search results for his specified query are presented.
- For each result, a concise overview of the associated product details is readily accessible
- Guest user clicks on 6th bike which he liked.



Screen 5: Product Detail Page

- **Scene:** Lukas clicks on one of the listings he admires.
- **Action:**
- The **Product Detail** page opens, displaying all the relevant information, including:

- Product name
- Description
- Images
- Price

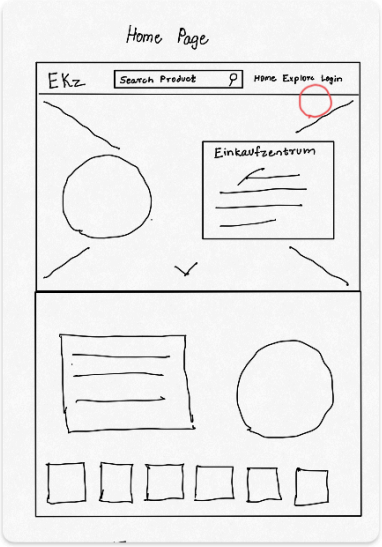


Screen 6: Login Prompt

- **Scene**: Lukas wants to chat with the seller and get the bike.
- **Action**: Since Lukas is a **guest**, he needs to **log in or sign up** to start the transaction or send a message to the seller.

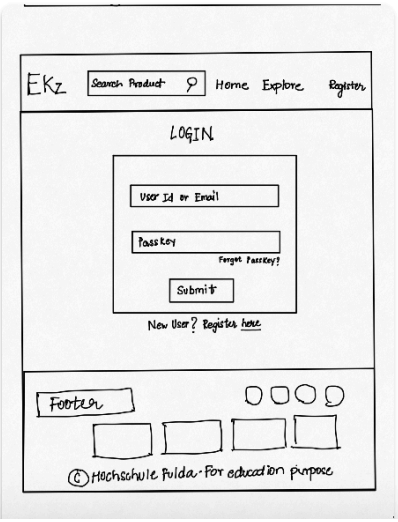
Use case 4

Admin Monitors Dashboard and Reports



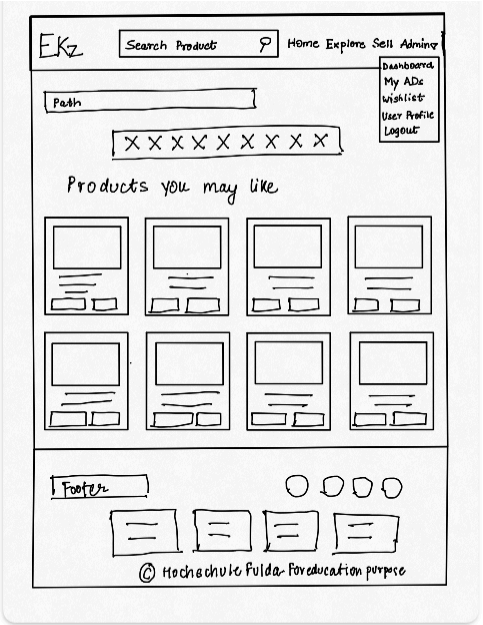
Screen 1: Homepage

- **Scene**: The admin opens the Einkaufzentrum website and lands on the homepage, just like any other user.
- **Action**: He clicks on the **"Login"** button.



Screen 2: Login Page

- **Scene**: The login form pops up.
- **Action**: The admin types in his **admin credentials** and logs in.



Screen 3: Explore Page

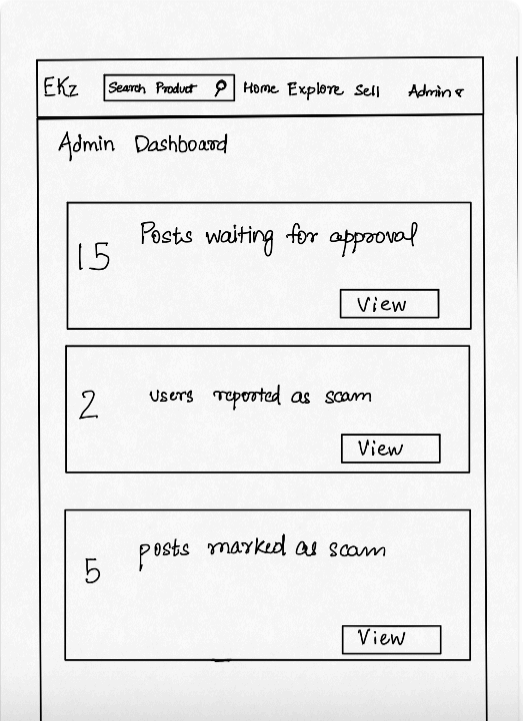
- **Scene**: After logging in successfully, the admin is greeted with the Explore page.
- **Action**: When clicked on admin keyword in navbar a dropdown appears with **"Dashboard"** option.

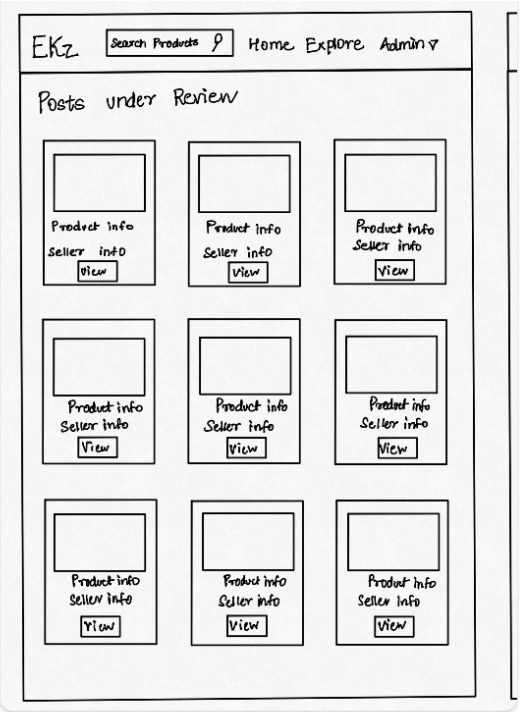
Screen 4: Admin Dashboard

- **Scene:** The **Admin Dashboard** page loads.
- **Action:** The dashboard displays three primary sections (cards or tabs):

*** Posts awaiting approval**
*** Users reported as scams**
*** Posts marked as scam/spam**

Each section contains a **"View All" button** or a similar action to manage and inspect items in detail.





Screen 5: Posts Pending Approval

- **Scene:** An administrator navigates to **Section 1**, *which displays posts awaiting approval.*
- **Action:**
- A new screen emerges, presenting a comprehensive list of **posts** submitted by sellers. Each post is meticulously detailed, accompanied by a **"View"** button for easy access.

Screen 6: Post Approval Detail View

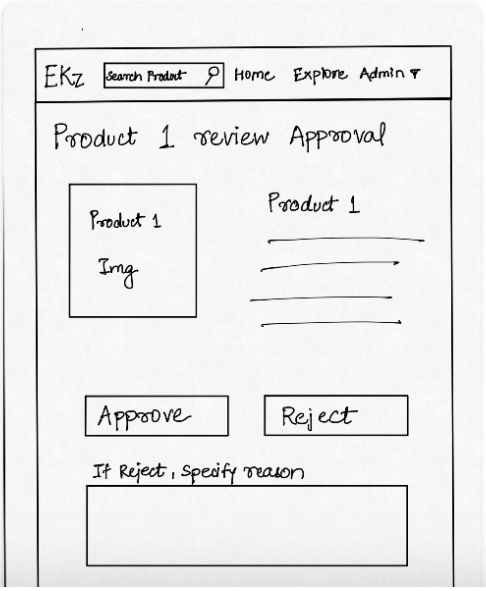
- **Scene:** An administrator clicks **"View"** on a specific post.
- **Action:**

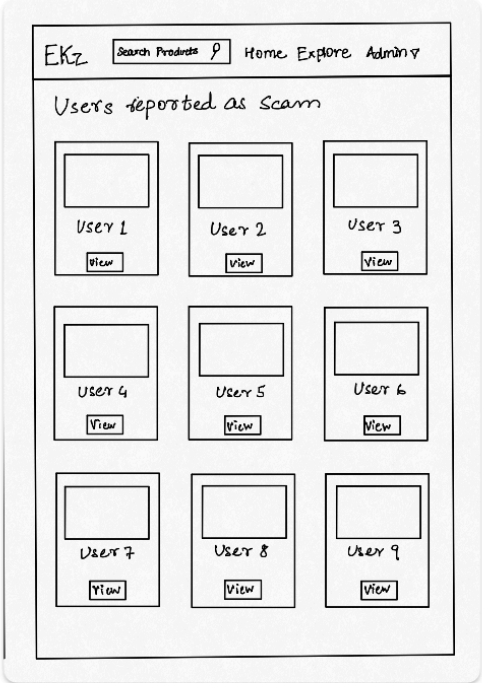
- The **Product Detail (Approval View)** page opens, displaying comprehensive post information such as title, image, description, and more.

- The administrator reviews the content and has two options:

* **Approve Post:** This action makes the post live.
* **Reject Post:** The administrator must provide a reason in a text field (e.g., "Incomplete details" or "Inappropriate content").

- Upon rejection, the seller has to **resubmit** the post.





Screen 7: Users Reported as Scams

- **Scene:** An administrator clicks on "**Section 2**: *Users reported as scams.*"
- **Action:**
- A new screen loads with **user cards** or **tab-like modules**, each containing:

* Brief user information
* Report reason or the number of posts marked as scams
* **"View"** button to inspect further

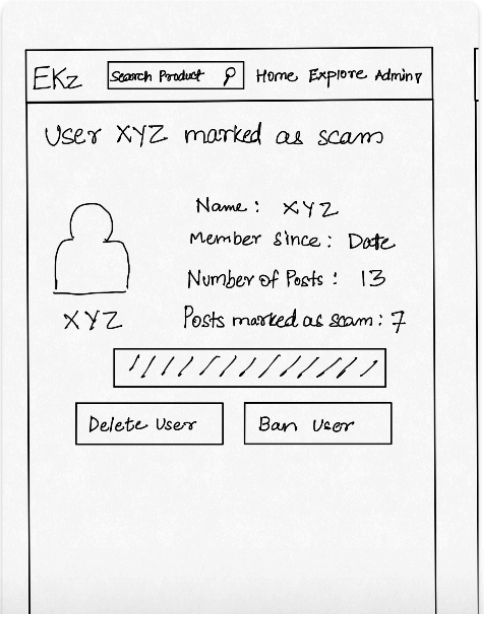
Screen 8: User Detail View (Scam Report)

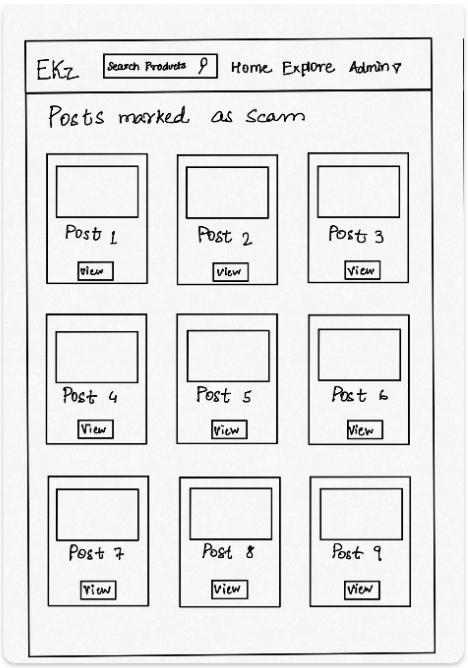
- **Scene:** An administrator clicks **"View"** on a reported user.
- **Action:**
- A new screen opens, displaying detailed user information:

* **Name**
* **Member since**
* **Total posts**
* **Posts marked as scam/spam**

- The administrator is provided with two action buttons:

* **Ban User**: Temporarily restrict account access for a relevant time period.
* **Delete User**: Permanently remove the user from the platform.





Screen 9: Posts Marked as Scam/Spam

- **Scene: Admin Clicks on Section 3 - *Posts Marked as Scam/Spam***
- **Action:**

- A new screen opens, displaying concise information about all posts marked as scam. This information is presented in a **tabbed or card-like** format, with a **"View"** button for further access.

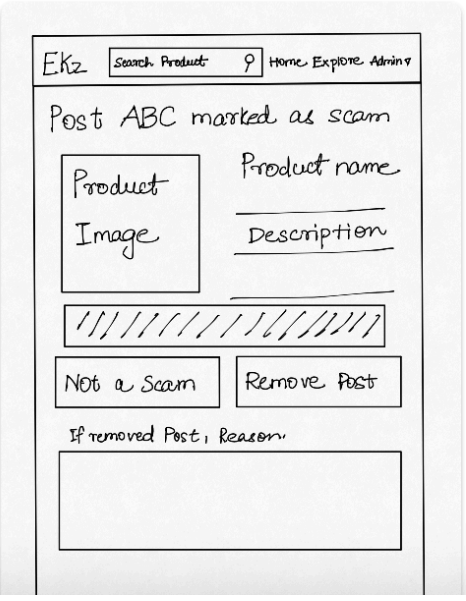
Screen 10: Scam Post Detail View

- **Scene:** An administrator clicks **"View"** on one of the scam-marked posts.
- **Action:**
- A detailed page opens, displaying the following information:

- Product name
- Image
- Description
- Scam context provided by the buyer

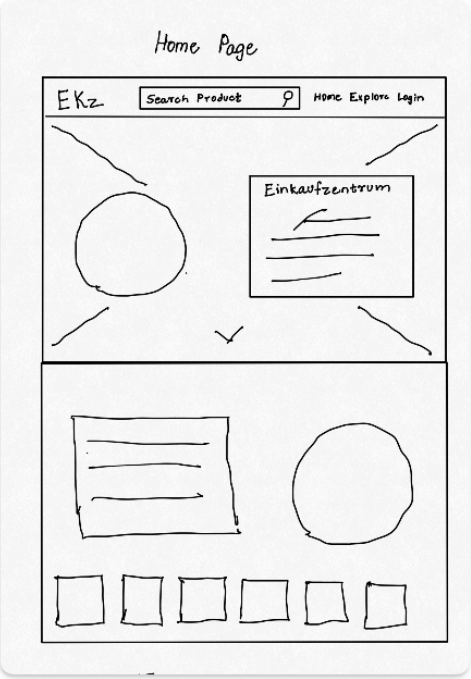
- The administrator reviews the post and its context and chooses one of two actions:

- **Remove Post**: This action requires a mandatory input field to **write a reason** (e.g., "Misleading content," "Duplicate listing," etc.).
- **Unmark as Scam**: This action restores the post if the report was deemed incorrect.



Use case 5

New user account creation.



Screen 1: Homepage

- **Scene:** Andrew Meier navigates to the homepage of the Einkaufzentrum website.
- **Action:** He clicks on the prominent **"Sign Up"** button situated at the top of the page.

Screen 2: Signup Page

- **Scene:** The signup form screen appears.
- **Action:**
- Andrew completes the following details:

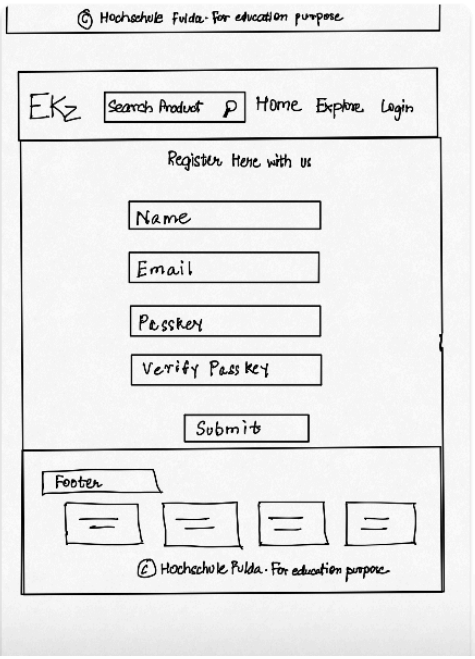
*** Full Name**

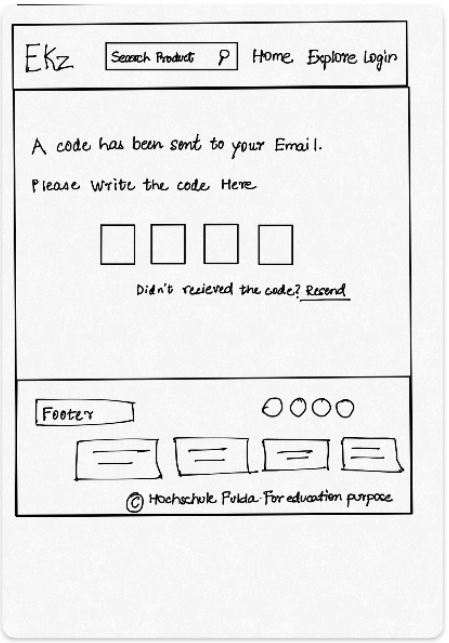
*** University Email**

*** Password**

*** Verify Password**

- He then clicks the **"Submit"** button to proceed.



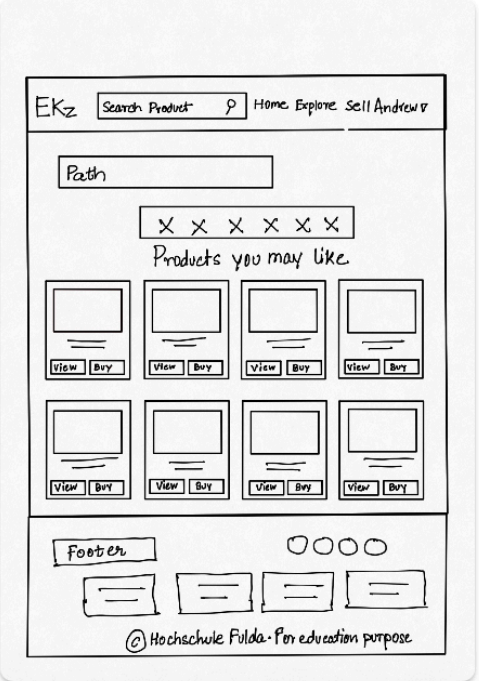


Screen 3: Email Verification Page

- **Scene:** Upon submitting the signup form, a new screen appears, prompting email verification.
- **Action:**
  - Andrew is informed that a **verification code** has been sent to his university email address.
  - He enters the received code into the provided input field.
  - He subsequently clicks **"Submit"** to verify his email and complete the registration process.

Screen 4: Explore Page

- **Scene: Upon Successful Email Verification**
- Upon successful email verification, Andrew is redirected to the Explore page.
- **Action:** As a registered user, he can now commence browsing, posting, or engaging with listings.



4. High-Level Architecture, Database Organization

i) Database Organization

The Fulda Einkaufszentrum application utilizes **MySQL 8.0** as its relational database management system, with **SQLAlchemy ORM** integrated into the Flask backend to manage database operations. The core tables within the database are structured as follows:

Table Name	Description	Key Fields
Users	All platform users (students, staff, admin)	user_id, name, email, role, status
Items	Listings for sale/exchange	item_id, title, description, price, category_id, seller_id, media_ref_url
Categories	Categories to group listings	category_id, name
Messages	Conversations between users	message_id, sender_id, receiver_id, content, timestamp
ApprovalQueue	Admin moderation of new listings	queue_id, item_id, status, admin_id
Reports	User-reported violations/inappropriate content	report_id, reporter_id, target_id, reason, status
LoginSessions	Tracks user login/logout sessions	session_id, user_id, login_time, logout_time
MediaReferences	Links to external media files	media_id, item_id, media_url, type

Notes:

- *All primary keys (PK) are auto-incremented integers.*
- *Foreign keys (FK) ensure referential integrity across related tables.*
- *Relationships such as one-to-many (e.g., **user** → **items**, **user** → **messages**) and many-to-one (e.g., item → category) are enforced via SQLAlchemy.*
- *Timestamps are stored in UTC and indexed for efficient querying.*
- *Admin tools access **Reports** and **ApprovalQueue** for moderation.*
- *Images and videos are referenced via **MediaReferences**, not stored in DB.*

- *All user content must pass through **ApprovalQueue** before going live.*

ii) Media Storage

Multimedia content, including images and videos, is stored externally on AWS S3 to optimize database performance and scalability. The MediaReference table contains URLs that link to these files, allowing the application to retrieve and display them efficiently without burdening the database. This approach was selected over database BLOB storage for its superior scalability and cost-effectiveness, particularly given the integration with the existing AWS infrastructure.

Messaging Architecture (Socket.IO Integration)

The Fulda Einkaufszentrum application supports asynchronous one-to-one messaging, with plans to introduce real-time messaging in future milestones. The architecture for real-time communication is built using:

- **Backend**: **Flask-SocketIO** (WebSocket support via eventlet or gevent).
- **Frontend**: **React** with **socket.io-client**.
- **Channels**: Messages are transmitted via private rooms (e.g., room = user1_user2).
- **Database**: Messages are persisted in the Messages table for historical access.

This setup enables:

- Real-time message delivery without polling.
- Efficient use of bandwidth and server resources.
- Future scalability for features like typing indicators or read receipts.

Example Workflow:

1. User A sends a message to User B via socket.io.
2. Flask-SocketIO receives the message, stores it in the database, and emits it to User B's socket.
3. User B's React client listens for new messages and updates the UI instantly.

iii) Search/Filter Architecture

The search and filter functionality is implemented using **SQLAlchemy** with standard SQL queries. Users can search for items based on keywords, categories, and price ranges. For example:

```
# SQLAlchemy example for item search

results = db.session.query(Item).filter(

   Item.title.ilike(f"%{keyword}%"),

   Item.description.ilike(f"%{keyword}%"),
```

```
    Item.category_id == category_id,

    Item.price <= max_price

).order_by(Item.created_at.desc()).all()
```

- **Keyword Search**: Applied to title and description fields using the LIKE operator.
- **Filters**: Applied to category_id and price fields.
- **Sorting**: Results can be ordered by recency or price (ascending or descending).

Future enhancements may include the adoption of MySQL's FULLTEXT indexing or integration with ElasticSearch for improved search performance and relevance ranking as the dataset grows.

iv) Custom APIs

The application provides a set of RESTful APIs built with Flask-RESTful, secured using JWT tokens stored in cookies. Key endpoints include:

Endpoint	Method	Description
/api/items	POST	Creates a new item listing (sent to ApprovalQueue).
/api/items/<category>	GET	Retrieves listings by category.
/api/messages/send	POST	Sends a message (saved and emitted via SocketIO).
/api/messages/thread/<user_id>	GET	Retrieves conversation history with a user.
/api/report	POST	Submits a report for a listing or user.
/api/approval	POST	Allows admins to approve or reject listings.

All API requests are authenticated via JWT to ensure secure access.

v) Significant Algorithms

Currently, no complex algorithms are implemented. However, future plans include:

- **Admin Prioritization**: Automatically prioritizing new listings based on posting time and category demand.
- **Search Optimization**: Introducing relevance ranking for search results based on keyword frequency in title and description.
- **Content Moderation**: Implementing a basic keyword filter to detect and flag inappropriate content in messages or listings.

vi) Changes in SW Tools and Frameworks

The following tools and frameworks have been integrated or planned to support the Fulda Einkaufszentrum app's functionality and scalability:

- **Flask-SocketIO**: Implements real-time, bidirectional messaging via WebSockets, enabling instantaneous message processing and delivery.
- **React with socket.io-client**: Integrates with the frontend to establish WebSocket connections, ensuring real-time message rendering in the user interface.
- **Flask-JWT-Extended**: Provides stateless, token-based authentication using JWT, enhancing security without server-side session management.
- **Postman**: Facilitates API testing and validation, ensuring endpoint reliability and adherence to specifications.
- **Flask-CORS**: Manages Cross-Origin Resource Sharing (CORS) to enable secure communication between the frontend and backend, hosted on separate domains.

Planned Enhancements:

- **Full-text Search or ElasticSearch**: To optimize search performance and relevance for large-scale datasets.
- **Celery**: To handle asynchronous background tasks, such as email notifications or content moderation, improving application responsiveness.
- **Content Filter**: To implement a keyword-based filtering mechanism for detecting and mitigating inappropriate content in messages and listings.

5. High-Level UML Diagrams

This UML class diagram models the core implementation classes of the Fulda Einkaufszentrum web application. It reflects an object-oriented (OO) approach, showing the main entities involved in the site's functionality such as user management, item listings, moderation, and messaging.

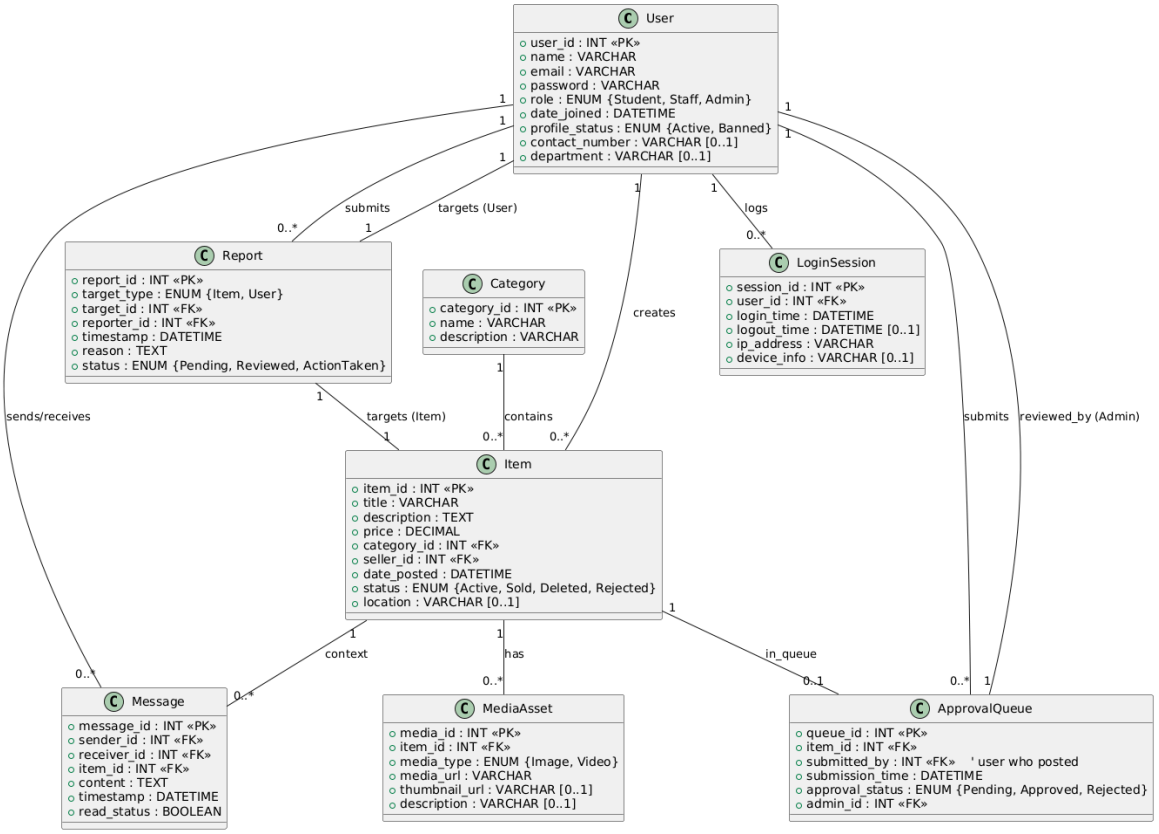
The three high-level UML diagrams show the structure and working of our Fulda Einkaufszentrum application. The Class Diagram presents the main data entities like User, Item, Message, and how they are related. The Component Diagram shows how the system is divided into frontend (React on S3), backend services (Flask), and external services (like S3 for media and a spam detection model), all connected through an Nginx reverse proxy. The Deployment Diagram explains how these parts are deployed on servers, showing user access via browser, frontend on S3, backend behind Nginx, and MySQL for data storage. These diagrams reflect a clear object-oriented and layered architecture.

Relationships (OO-Oriented)

- A User can create multiple Items.
- Each Item is associated with one Category.
- An Item can have multiple MediaAssets (images/videos).
- Messages are exchanged between Users, and are tied to a specific Item.
- A User can submit Reports against Users or Items.

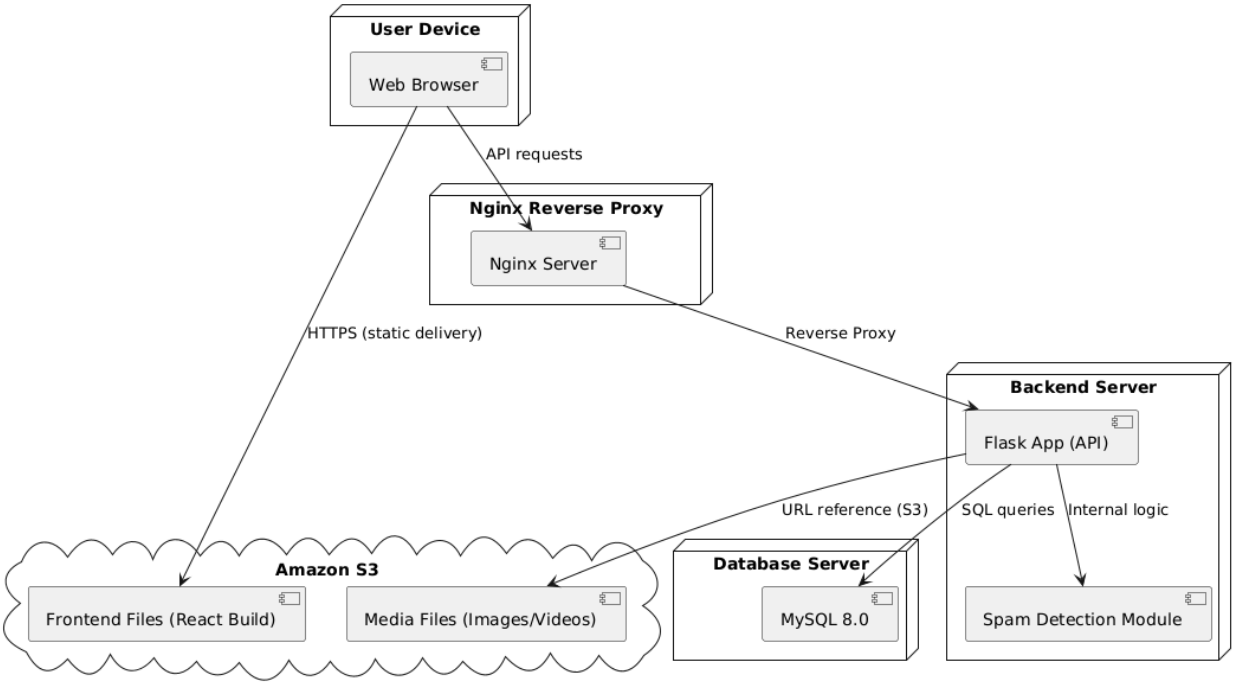
- An Item can be in one ApprovalQueue, and is submitted by a User, then reviewed by an Admin.
- Users have multiple LoginSessions, which record their login activity.

UML Class Diagrams

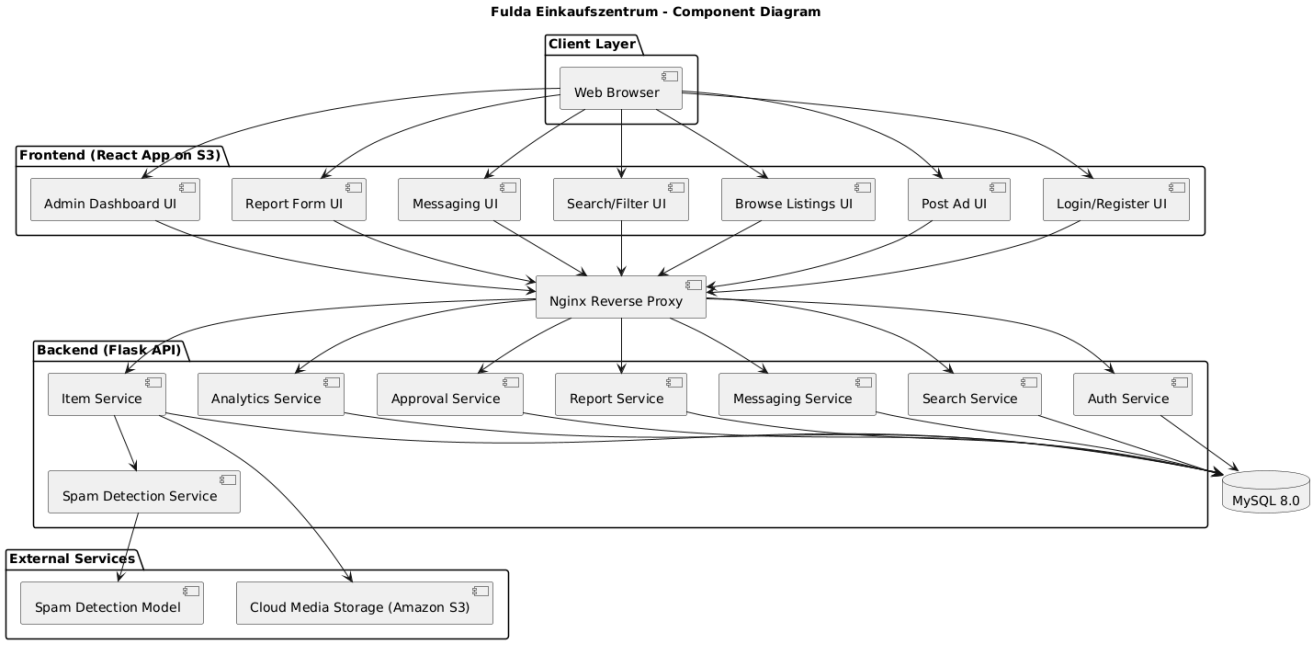


UML Component and Deployment Diagrams

Component Diagram:



Deployment Diagram:



6. Identify Actual Key Risks

Skills Risks

- We are a team of expertise in different domains, even though all of us do not have experience in all the sides of the project completion, we are learning from each other's expertise in that area along with self learning as well to make this project become successful.

Schedule Risks

- We have added each and every milestone's task based on priority once it is given to avoid any scheduling risk in the project completion.

Technical Risks

- **Exceeding Free Tier Limits (**AWS Cloud Deployment)
  - The AWS Free Tier offers 750 hours per month for t2.micro/t3.micro EC2 and RDS instances, and 5GB of S3 storage. If you exceed these limits, you will incur charges.
  - **Mitigation:**

  Use AWS Budgets to set up alerts for billing thresholds. Monitor resource usage regularly via the AWS Billing Dashboard. Keep usage within limits by stopping EC2 and RDS instances when not in use (e.g., overnight or weekends).
- **S3 Storage and Request Limits**
  - Free S3 usage includes 5GB storage and limited PUT/GET requests. Uploading too many product images or files may quickly exceed the threshold.
  - **Mitigation:**

Compress images before upload and apply lifecycle rules to automatically delete old or unused images. Use S3's object expiry policies to manage storage efficiently.
- **Unexpected Billing Due to Non-Free Services (**AWS Cloud Deployment)
  - You may accidentally launch services or instance types (e.g., t2.medium or multi-AZ RDS) that aren't part of the Free Tier.
  - **Mitigation:**

  Always select "Free Tier eligible" options while provisioning. Review configurations before launch and enable billing alerts. Use the AWS Cost Explorer to detect anomalies.
- **Image Link Management in Database**
  - **Issue**: Storing only links to AWS S3 for product images can lead to broken references if the S3 object is deleted or the URL format changes.
  - **Mitigation:**

Need to Regularly validate links in the database either by automated cron job or by scheduled query checks and need to consider storing metadata like upload date for better tracking.

Teamwork Risks

- We as a team have expertise in different domains so as a collaborative team, we haven't encountered any risk. We use Trello as our collaborative tool to divide and assign tasks and google meet for our weekly online meeting.

Legal/Content Risks

- We are using few open source libraries in our development so whenever there is a need to address the owning party of those packages, we planned to include them in the project repository as well to avoid licensing/copyright issues.

7. Project Management

Since starting the project, we started using the Trello application to organize and manage all the tasks we planned in the project. We created a ***"GDSD Project Team 4 Fulda Summer 2025"*** board in the Trello app and added different task lists like TODO, In Progress, Completed and Done & Archived lists. Initially, we created tasks in the TODO list and once it was assigned for a team member after discussion, it will be moved to the In Progress list when the teammate started working on it. After the task completion it will be marked as completed and moved to the 'Completed' task list and once after the milestone is submitted we move it to the 'Done & Archived' task list for easy understanding instead of having too many open tasks on the board.

As of milestone 2, we haven't started on technical development so for now we added only Milestone 0,1,2 as labels in the project task list. We planned to assign labels as frontend, backend and others in the future task items for easy grouping. We planned to use the trello app throughout our entire project completion since its helpful for us to remember the tasks we did each week and To do which need to be done by us. This in turn helped us to update our weekly timesheet as well. Here is how our recent trello board looks as of this current milestone.

